

Technical Interview Tips & System Design Basics

CareerForge Knowledge Base · Volume 4

2026 context: Technical interviewing is mid-transition. AI-assisted coding rounds are becoming common at some companies, while others have doubled down on proctored, AI-banned formats. Knowing which one you're walking into changes your prep -- always ask the recruiter in advance.

1. The Three Interview Formats You'll Encounter

Format	What it looks like	How to prepare
AI-banned, proctored	Live coding on a locked-down platform or screen-share; algorithm-focused (DSA)	Practice solving problems unaided, out loud, under time pressure
AI-assisted round	You're given an approved AI assistant (e.g. in a shared coding environment) and graded on how well you use it	Practice verifying AI output, catching its mistakes, and explaining your reasoning -- refusing to use the tool is itself a mistake here
Take-home / open-ended	A project or system design discussion, AI use usually assumed and not policed	Focus on what you ship and your ability to defend design decisions and trade-offs

2. Coding Round Tips (DSA still matters)

Despite predictions of its death, data-structures-and-algorithms interviewing remains standard at most companies in 2026 -- it's language-neutral and gives a standardized way to compare candidates across very different backgrounds. What has changed is emphasis: fewer obscure "trick" puzzles, more practical string/array/graph problems resembling real code.

- Clarify constraints and edge cases out loud before writing any code -- input size, duplicates, empty input, sorted vs unsorted.
- State a brute-force approach first, even if you know it's not optimal -- it shows you can produce a working baseline, then improve it.
- Narrate your thinking continuously; a silent candidate is much harder to evaluate than one who thinks out loud, even with the same final answer.
- Always state the time and space complexity of your final solution without being asked.
- Test your solution against at least one edge case before declaring it done.

Priority areas to review

- Arrays, strings, hashmaps -- the most common building blocks in practical interview questions.
- Trees and graphs (BFS/DFS, traversal patterns) -- especially common in mid-to-senior loops.
- Two pointers, sliding window, and binary search patterns -- these solve a large fraction of "medium" difficulty problems.
- Basic dynamic programming patterns (top-down memoization is usually enough; exotic DP is rarer now).

3. System Design: A Repeatable Framework

System design interviews reward structure over memorized architectures. Use the same five-step framework every time:

- **1. Clarify requirements.** Functional (what must it do) and non-functional (scale, latency, consistency vs availability trade-offs). Never start drawing boxes before this.
- **2. Estimate scale.** Rough numbers for users, requests/sec, data volume/growth -- back-of-envelope math, not precision.
- **3. High-level design.** Draw the major components and how data flows between them before going deep on any one part.
- **4. Deep dive.** Pick 1-2 components the interviewer cares most about and go deep: data model, API contract, failure handling.
- **5. Trade-offs and bottlenecks.** Proactively name what would break at 10x scale, and what you'd change -- this is where senior candidates separate themselves in 2026's higher-bar interviews.

Common building blocks to know cold

- Load balancers, horizontal scaling, and stateless services
- Caching layers (Redis/Memcached) and cache invalidation strategies
- Database replication, sharding, and read/write separation
- Message queues for async processing and decoupling (Kafka, SQS-style)
- CDNs for static content delivery

The 2026 shift: AI/LLM system design

System design loops increasingly include AI/LLM infrastructure questions -- even for general software engineering roles, not just ML positions. Be ready to discuss:

- Designing a RAG (Retrieval-Augmented Generation) pipeline: chunking strategy, vector store choice, retrieval + reranking, and how you'd keep the knowledge base fresh
- Latency and cost trade-offs when calling an LLM API (batching requests, caching common queries/responses, streaming partial output to the user)
- Failure modes specific to LLM systems: hallucination mitigation, rate limits, fallback to a smaller/cheaper model, prompt injection from untrusted input
- When to fine-tune vs. use retrieval vs. use a bigger base model -- and how to justify the choice on cost and latency grounds

4. Mock Interview Practice

- Practice with a real listener whenever possible -- narrating to an empty room is not the same skill as narrating under social pressure.
- Time yourself: aim for a working (not necessarily optimal) coding solution in 20-25 minutes, leaving time to discuss trade-offs.
- Record yourself occasionally -- most candidates are surprised by how much they say "um" or go silent while thinking.

- After each mock, write down the one thing that went worst and drill that specific gap before the next one.

5. Quick Pre-Interview Checklist

- ■ Confirmed with the recruiter whether AI tools are allowed in this specific round
- ■ Reviewed core DSA patterns relevant to the role (not just memorized solutions)
- ■ Can recite the 5-step system design framework without looking it up
- ■ Prepared 2-3 clarifying questions to ask before diving into any design problem
- ■ Practiced narrating thought process out loud, not just solving silently
- ■ Ready to discuss trade-offs and "what breaks at 10x scale" for any design

This guide reflects widely-observed hiring practices and 2025-2026 industry trends. It is general career guidance, not a guarantee of interview or offer outcomes -- always adapt advice to your specific role, industry, and location.